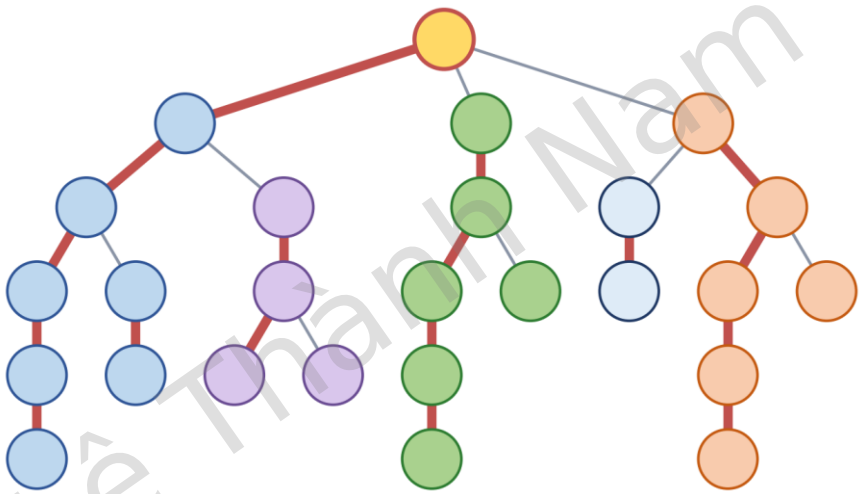
Tổng hợp và biên soạn: Lê Thành Nam

TÀI LIỆU BỒI DƯỠNG HỌC SINH GIỎI TIN HỌC

THUẬT TOÁN TRÊN CÂY

Dễ hiểu cho người bắt đầu từ con số 0

TẬP 5 · PHÂN RÃ CÂY



Lê Thành Nam

Chia nặng nhẹ · Phân rã trọng tâm · Cây trọng tâm · Chia theo đường dài · Chia căn · Mo trên cây

Chương 1. CHIA NẶNG NHẸ
Chương 2. PHÂN RÃ TRỌNG TÂM
Chương 3. CHIA THEO ĐƯỜNG DÀI
Chương 4. CHIA CĂN TRÊN CÂY

LỜI NÓI ĐẦU

Trên mảng, ta có cả một kho công cụ: cây phân đoạn, mảng cộng dồn, hai con trỏ, chia khối. Trên cây, những công cụ ấy không dùng trực tiếp được, vì một đường đi hay một tập đỉnh cùng tầng không phải là một đoạn. Cuốn sách này dạy các cách **phân rã cây**: cắt cây thành những mảnh có cấu trúc đơn giản (chuỗi, thành phần nhỏ, khối) sao cho mọi câu hỏi chỉ chạm vào ít mảnh, rồi dùng lại kho công cụ của mảng trên từng mảnh.

Đây là **Tập 5: Phân rã cây** của bộ sách về thuật toán trên cây, tiếp nối Tập 1 (Nền tảng), Tập 2 (Tổ tiên chung gần nhất), Tập 3 (Quy hoạch động trên cây) và Tập 4 (Kỹ thuật gộp). Em cần đã quen với: BFS và thứ tự trước, kích thước cây con, LCA, cây phân đoạn và BIT, ý tưởng con nặng của Tập 4. Những kiến thức ấy được nhắc lại ở Phần mở đầu.

Giống các tập trước, mỗi bài gồm hai phần:

- **Phần A – Lí thuyết:** định nghĩa, định lí kèm chứng minh từng ý, thuật toán, chạy tay từng bước trên hình (phần lớn trên cây mẫu T14), code mẫu, độ phức tạp, bẫy thường gặp.
- **Phần B – Bài vận dụng:** một bài toán theo khuôn đề thi, giải đầy đủ: ý tưởng ngây thơ với sơ đồ khối và chương trình vét cạn, chỉ ra bế tắc, áp dụng lí thuyết để phá bế tắc, chạy tay, chương trình hoàn chỉnh, mở rộng và luyện tập.

Sách gồm **Phần mở đầu** và **11 bài học chia thành 4 chương**:

- **Chương 1. Chia nặng nhẹ:** chuỗi nặng, nhảy chuỗi $O(\log^2 n)$; trọng số trên cạnh; đường đi và cây con cùng lúc, lan truyền lười, đổi gốc.
- **Chương 2. Phân rã trọng tâm:** trọng tâm, cây trọng tâm; đếm đường đi theo độ dài và trọng số; truy vấn gần nhất trực tuyến.
- **Chương 3. Chia theo đường dài:** con sâu, kế thừa con trỏ, QHĐ theo độ sâu $O(n)$; tổ tiên thứ k $O(1)$.
- **Chương 4. Chia căn trên cây:** tầng lớn và tầng nhỏ, xây lại định kì; thuật toán Mo trên cây.

Các bài vận dụng gồm những bài kinh điển: **CSES Path Queries II** (Bài 1), **SPOJ QTREE** (Bài 2), **CSES Fixed-Length Paths I, II** (Bài 4, 5), **IOI 2011 Race** (Bài 6), **Codeforces 342E Xenia and Tree** (Bài 7), **Codeforces 1009F Dominant Indices** (Bài 8), **POI 2014 Hotels** (Bài 9), **SPOJ COT2** (Bài 11). Mọi chương trình đều đã được đối chiếu với chương trình vét cạn trên hàng trăm bộ dữ liệu ngẫu nhiên và chạy thử với dữ liệu lớn nhất; mọi ví dụ trong sách đều được chạy lại bằng chính các chương trình đó.

Chương trình viết bằng C++17, đọc và ghi tệp theo kiểu "có tệp thì đọc tệp", dùng được với Themis lẫn các trang chấm trực tuyến. Mọi chương trình đều không có chú thích bên trong; các dòng quan trọng được giải thích ngay bên dưới.

Chúc các em và các thầy cô học tập say mê, dạy tốt và đạt kết quả cao!

Lê Thành Nam

Lê Thành Nam

CÁCH HỌC VỚI TÀI LIỆU NÀY

Mỗi bài học có cùng một cấu trúc:

Phần	Nội dung
Tóm tắt bài học	Khung tóm tắt ở đầu bài: kiến thức chính, khuôn cài đặt, độ phức tạp, khi nào dùng, bài vận dụng.
Phần A – Lí thuyết	Định nghĩa, tính chất, định lí kèm chứng minh từng ý; thuật toán; chạy tay từng bước nhỏ trên hình; code mẫu; độ phức tạp; bẫy; ghi nhớ.
Phần B – Bài vận dụng	Đề bài như đề thi thật: dữ liệu vào, kết quả, giới hạn, ví dụ.
B1. Ý tưởng ngây thơ	Cách làm ai cũng nghĩ ra: sơ đồ khối, chạy tay, chương trình vét cạn, đếm số phép tính.
Bế tắc	Chỉ ra chính xác vì sao cách ngây thơ quá chậm hoặc quá tốn bộ nhớ.
B2. Phá bế tắc	Áp dụng lí thuyết Phần A, từng bước, chạy tay trên hình.
B3. Chương trình hoàn chỉnh	Chương trình C++ đã kiểm chứng, giải thích các dòng quan trọng.
B4. Mở rộng, Luyện tập	Các biến thể thường gặp; bài luyện thêm và câu tự kiểm tra có đáp số.

Các khung màu trong sách có ý nghĩa cố định:

Khung	Ý nghĩa
TÓM TẮT BÀI HỌC	Tóm tắt toàn bộ bài ở ngay đầu bài.
ĐỊNH NGHĨA	Định nghĩa chính xác một khái niệm.
TÍNH CHẤT / ĐỊNH LÍ	Một khẳng định quan trọng, luôn có chứng minh đi kèm.
THUẬT TOÁN	Các bước của một thuật toán.
HỘP CÔNG CỤ C++	Giới thiệu một công cụ C++ khi nó được dùng lần đầu.
ĐẾM SỐ PHÉP TÍNH	Phân tích độ phức tạp: vì sao chậm, vì sao nhanh.
BẾ TẮC	Chỗ cách làm ngây thơ không vượt qua được.
BẪY THƯỜNG GẶP	Các lỗi hay gặp.
GHI NHỚ	Tóm tắt cuối phần lí thuyết.

Quy ước màu trong các hình được giữ nguyên trong cả cuốn sách: **đỉnh vàng viền đỏ** là đỉnh đang xét; **đỉnh xanh lá** là đỉnh đã xử lí xong, hoặc đỉnh được chọn trong lời giải; **đỉnh xanh dương nhạt** là đỉnh bình thường; **đỉnh đỏ nhạt** là đỉnh bị loại (bị cắt tỉa, hoặc là đỉnh đỏ trong bài Xenia); **gốc** tô xanh đậm. Ở Chương 1, mỗi chuỗi nặng có một màu riêng; **cạnh đỏ đậm** là cạnh nặng (hoặc cạnh nối với con sâu); **cạnh cam đậm** là đường đi đang xét. Hàng ô bên dưới cây là mảng sau khi trải cây (pos, Euler); khung màu quanh một nhóm ô là một đoạn được hỏi.

Lê Thành Nam

Lời khuyên khi học: Hãy đọc với giấy và bút bên cạnh. Gặp một hình chạy tay, hãy che phần lời giải lại, tự làm bước đó rồi mới đọc. Gặp một định lí, hãy tự vẽ một cái cây khác và kiểm tra định lí trên cây của mình. Chỉ khi tự viết lại được chương trình và tự giải thích được vì sao nó đúng mà không nhìn sách, em mới thật sự nắm được bài.

Lê Thành Nam

MỤC LỤC

LỜI NÓI ĐẦU **2**
CÁCH HỌC VỚI TÀI LIỆU NÀY **3**
PHẦN MỞ ĐẦU. PHÂN RÃ CÂY VÀ BẢN ĐỒ TẬP 5 ..... **6**
CHƯƠNG 1. CHIA NẶNG NHẸ **9**
Bài 1. Chia nặng nhẹ (HLD) 10
Bài 2. Trọng số trên cạnh 29
Bài 3. Đường đi, cây con và lan truyền lưới 43
CHƯƠNG 2. PHÂN RÃ TRỌNG TÂM **58**
Bài 4. Trọng tâm và phân rã trọng tâm 59
Bài 5. Đếm trong đoạn độ dài 74
Bài 6. Trọng số cạnh và mảng theo khoảng cách 86
Bài 7. Cây trọng tâm cho truy vấn trực tuyến 97
CHƯƠNG 3. CHIA THEO ĐƯỜNG DÀI **110**
Bài 8. Chia theo đường dài 111
Bài 9. QHĐ hai mảng ngược chiều 125
CHƯƠNG 4. CHIA CĂN TRÊN CÂY **137**
Bài 10. Chia căn theo độ sâu và theo thời gian 138
Bài 11. Thuật toán Mo trên cây 151
TỔNG KẾT: CHỌN CÁCH PHÂN RÃ **163**

Lê Thành Nam

PHẦN MỞ ĐẦU

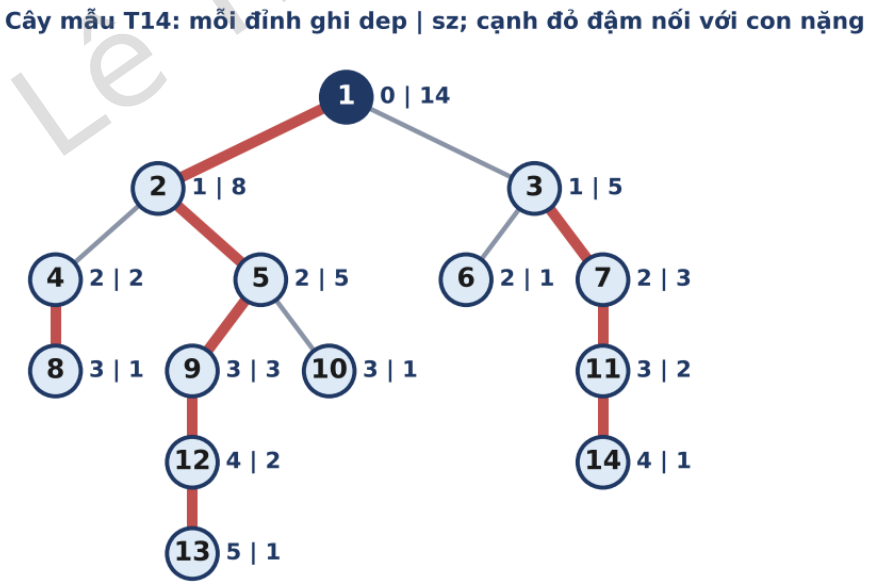
PHÂN RÃ CÂY VÀ BẢN ĐỒ TẬP 5

Phần này nhắc lại các kí hiệu từ các tập trước, giải thích "phân rã cây" nghĩa là gì, bốn cách cắt cây trong cuốn sách, lập luận chung đằng sau chúng, và vẽ bản đồ bốn chương.

1. Nhắc lại

Cây được treo ở **gốc 1**. Các kí hiệu dùng xuyên suốt:

Kí hiệu	Ý nghĩa	Ở đâu
par[v], dep[v], ord	cha; độ sâu (gốc ở độ sâu 0); thứ tự BFS	Tập 1
T(v), sz[v]	cây con gốc v; số đỉnh của nó	Tập 1
tin[v], ver[]	thứ tự trước; T(v) = ver[tin[v] .. tin[v] + sz[v] − 1]	Tập 1
LCA(u, v)	tổ tiên chung gần nhất	Tập 2
big[v], cạnh nhẹ	con có sz lớn nhất; cạnh xuống các con khác	Tập 2, 4
cây phân đoạn, BIT	hỏi và sửa trên đoạn của mảng trong O(log n)	Tập 1



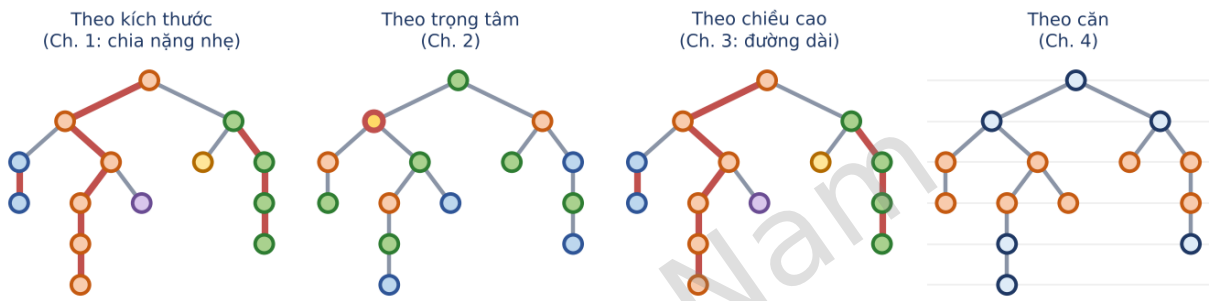
Hình 0.1. *Cây mẫu T14*

Lê Thành Nam

Phần lớn các ví dụ chạy tay dùng lại cây T14 (danh sách cạnh: 1 2, 1 3, 2 4, 2 5, 3 6, 3 7, 4 8, 5 9, 5 10, 7 11, 9 12, 12 13, 11 14). Khi cần giá trị trên đỉnh, ta dùng a = (5, 3, 8, 2, 7, 4, 1, 6, 9, 2, 5, 3, 4, 6) cho các đỉnh 1..14; khi cần trọng số cạnh, trọng số của cạnh (par[v], v) là w = (4, 7, 2, 6, 3, 5, 8, 1, 9, 2, 7, 3, 6) cho v = 2..14.

2. Phân rã cây là gì?

Mảng dễ hơn cây: trên mảng có cây phân đoạn, mảng cộng dồn, hai con trỏ, chia khối. "Phân rã cây" là cắt cây thành **các mảnh có cấu trúc đơn giản** (đường đi, thành phần nhỏ, khối) sao cho mọi câu hỏi trên cây chỉ chạm vào **ít** mảnh. Cuốn sách trình bày bốn cách cắt:



Hình 0.2. *Bốn cách cắt cây T14*

- **Theo kích thước (Chương 1).** Mỗi đỉnh nối dài chuỗi bằng con **nhiều đỉnh nhất**. Mọi đường đi chỉ cắt O(log n) chuỗi; xếp các chuỗi liên tiếp vào một mảng là dùng được cây phân đoạn.
- **Theo trọng tâm (Chương 2).** Xóa đỉnh mà mọi mảnh còn lại không quá một nửa, rồi làm tiếp trên từng mảnh. Sau log n tầng, mỗi đường đi đi qua đúng một "trọng tâm cao nhất" của nó.
- **Theo chiều cao (Chương 3).** Mỗi đỉnh nối dài chuỗi bằng con **sâu nhất**. Đường đi có thể cắt √n chuỗi, nhưng tổng độ dài chuỗi là n: QHĐ theo độ sâu chỉ tốn O(n).
- **Theo căn (Chương 4).** Tách đối tượng lớn và nhỏ theo ngưỡng √n, gom thao tác thành lô, hoặc sắp câu hỏi để hai con trỏ đi ít nhất.

3. Một lập luận chung: "một nửa"

> **NHÂN ĐÔI, CHIA ĐÔI**
> Nếu mỗi lần một đối tượng "trả phí" thì một đại lượng gắn với nó **gấp đôi** (hoặc **giảm một nửa**), và đại lượng ấy nằm giữa 1 và n, thì đối tượng trả phí **không quá $\log_2 n$ lần**.

Tập 4 đã dùng bổ đề này cho gộp nhỏ vào lớn. Ở tập này: đi qua một cạnh nhẹ thì sz hơn gấp đôi (Chương 1); đi xuống một mức của cây trọng tâm thì kích thước thành phần giảm ít nhất một nửa (Chương 2). Chương 3 thay "gấp đôi" bằng một lập luận "trả phí theo chuỗi" (mỗi chuỗi trả đúng một lần, tổng độ dài n), còn Chương 4 thay bằng phép cân bằng B = n/B.

Lê Thành Nam

4. Chọn kỹ thuật

Đề hỏi	Kỹ thuật	Bài
cập nhật và hỏi (max, tổng) trên đường đi, cây con	chia nặng nhẹ + cây phân đoạn	1, 2, 3
đếm hoặc tối ưu trên **mọi** đường đi (độ dài, trọng số)	phân rã trọng tâm	4, 5, 6
khoảng cách tới tập đỉnh thay đổi, trực tuyến	cây trọng tâm	7
QHĐ trên mọi cây con theo độ sâu	chia đường dài	8, 9
hai loại thao tác "không cùng thứ tự", hoặc câu hỏi không gộp được	chia căn, Mo trên cây	10, 11

5. Bản đồ Tập 5



Hình 0.3. *Bốn chương*

- **Chương 1 (Bài 1–3):** chuỗi nặng, nhảy chuỗi; trọng số trên cạnh; đường đi và cây con cùng lúc, lan truyền lười, đổi gốc.
- **Chương 2 (Bài 4–7):** trọng tâm, cây trọng tâm; đếm đường đi độ dài k, trong đoạn; trọng số và tối ưu; truy vấn gần nhất trực tuyến.
- **Chương 3 (Bài 8–9):** con sâu, kế thừa con trỏ, tổ tiên thứ k O(1); QHĐ hai mảng ngược chiều.
- **Chương 4 (Bài 10–11):** chia căn theo độ sâu và theo thời gian; thuật toán Mo trên cây.

> **KHÔNG ĐỆ QUY**
> Như các tập trước, mọi duyệt cây đều không đệ quy: BFS, thứ tự trước bằng ngăn xếp, phân rã trọng tâm bằng ngăn xếp các thành phần. Chỉ cây phân đoạn lan truyền lười ở Bài 3 dùng đệ quy có chủ ý, vì độ sâu chỉ là log n.

Lê Thành Nam

6. Đối chiếu chương trình

Mọi chương trình chính đã được đối chiếu với chương trình trâu trên 500 bộ dữ liệu nhỏ ngẫu nhiên (cây có sáu dạng: ngẫu nhiên, đường thẳng, hình sao, "con sâu", cây sâu, nhị phân), và chạy thử với dữ liệu lớn nhất; thời gian đo được ghi trong từng bài. Mọi ví dụ trong sách đều được chạy lại bằng chính các chương trình đó.

Lê Thành Nam

CHƯƠNG 1. CHIA NẶNG NHẸ

Cắt cây thành các chuỗi đi xuống, mỗi chuỗi nằm liên tiếp trong một mảng. Vì mỗi cạnh nhẹ làm cây con lớn gấp đôi, mọi đường đi chỉ cắt qua O(log n) chuỗi: một câu hỏi trên đường đi trở thành O(log n) câu hỏi trên đoạn.

TRONG CHƯƠNG NÀY

Bài 1. Chia nặng nhẹ (HLD) *(vận dụng: max trên đường đi (CSES Path Queries II))*

Bài 2. Trọng số trên cạnh *(vận dụng: cạnh lớn nhất trên đường đi (SPOJ QTREE))*

Bài 3. Đường đi, cây con và lan truyền lười *(vận dụng: cộng và tính tổng trên đường đi, cây con)*

Lê Thành Nam

BÀI 1 Chia nặng nhẹ (HLD)

Bài vận dụng: Max trên đường đi (CSES Path Queries II)

TÓM TẮT BÀI 1. CHIA NẶNG NHẸ (HLD)

Chủ đề: Chia nặng nhẹ

Kiến thức chính:

- Con nặng = con có sz lớn nhất; cạnh nhẹ (v, u): sz[u] < sz[v]/2 ⇒ ≤ $\log_2 n$ cạnh nhẹ trên đường lên gốc.
- Thứ tự trước, con nặng trước: mỗi chuỗi và mỗi cây con là một đoạn của mảng pos.
- Nhảy chuỗi: đầu có head sâu hơn lấy đoạn [pos[head], pos[x]] rồi lên par[head]; cuối cùng đoạn giữa hai đầu.

Khuôn cài đặt:

```
while (head[x] != head[y]) {
    if (dep[head[x]] < dep[head[y]]) swap(x, y);
    res = max(res, hoi(pos[head[x]], pos[x]));
    x = par[head[x]];
}
if (dep[x] > dep[y]) swap(x, y);
res = max(res, hoi(pos[x], pos[y]));
```

Độ phức tạp: $O(\log^2 n)$ mỗi câu hỏi đường đi

Dùng khi: Câu hỏi và cập nhật trên đường đi

Bài vận dụng: Max trên đường đi (CSES Path Queries II) [DUONGMAX]

Học xong bài này, em sẽ:

- ✓ Hiểu vì sao muốn "trải cây thành mảng" để dùng cây phân đoạn cho các câu hỏi **trên đường đi**.
- ✓ Nắm định nghĩa **con nặng, cạnh nhẹ, chuỗi nặng, head[v]** và chứng minh được: mọi đường đi lên gốc qua **không quá $\log_2 n$ cạnh nhẹ**.
- ✓ Đánh số **pos** sao cho mỗi chuỗi và mỗi cây con là một đoạn liên tiếp; cài đặt không đệ quy.
- ✓ Cài đặt thuật toán **nhảy chuỗi**, chứng minh nó đúng và chạy trong $O(\log^2 n)$.

PHẦN A LÍ THUYẾT

A1. Từ mảng lên cây

Trên **mảng**, bài toán "đổi một phần tử, hỏi giá trị lớn nhất của một đoạn" được cây phân đoạn giải trong O(log n) mỗi thao tác. Trên **cây**, câu hỏi tương ứng là "giá trị lớn nhất trên **đường đi** từ x đến y". Đường đi trên cây không phải đoạn của một mảng nào cả, trừ khi cây là một đường thẳng.

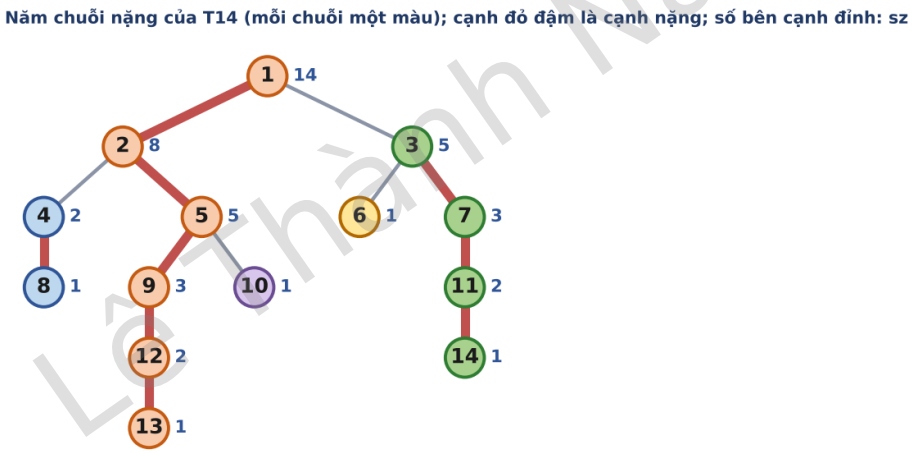
Lê Thanh Nam

Ý tưởng của **chia nặng nhẹ** (Heavy-Light Decomposition, viết tắt HLD): cắt cây thành nhiều **đường đi xuống** rời nhau (gọi là **chuỗi**), xếp mỗi chuỗi vào một đoạn liên tiếp của một mảng chung. Một đường đi x – y bị các chuỗi cắt thành vài mẩu, mỗi mẩu là một đoạn của mảng, hỏi bằng cây phân đoạn. Mọi thứ phụ thuộc vào câu hỏi: **đường đi bị cắt thành bao nhiêu mẩu?** Cắt khéo thì chỉ O(log n) mẩu.

A2. Con nặng, cạnh nhẹ, chuỗi nặng

ĐỊNH NGHĨA

- Cây treo ở gốc 1; sz[v] là số đỉnh của cây con T(v).
- **Con nặng** big[v] của đỉnh trong v: con có sz lớn nhất (bằng nhau thì chọn một con bất kì). Cạnh (v, big[v]) là **cạnh nặng**; mọi cạnh khác từ v xuống con là **cạnh nhẹ**; các con khác là **con nhẹ**.
- **Chuỗi nặng**: một đường đi xuống chỉ gồm cạnh nặng, kéo dài tối đa. Đỉnh trên cùng của chuỗi chứa v là **head[v]**.



Hình 1.1. *Năm chuỗi nặng của cây T14*

Trên Hình 1.1: sz[2] = 8 > sz[3] = 5 nên big[1] = 2; tiếp tục big[2] = 5, big[5] = 9, big[9] = 12, big[12] = 13: chuỗi cam 1 – 2 – 5 – 9 – 12 – 13. Đỉnh 3 là con nhẹ của 1, nó bắt đầu chuỗi xanh lá 3 – 7 – 11 – 14. Các chuỗi còn lại: 4 – 8, chuỗi một đỉnh {10} và {6}. Vậy head[13] = 1, head[14] = 3, head[8] = 4, head[10] = 10.

TÍNH CHẤT 1

Mỗi đỉnh thuộc **đúng một** chuỗi; mỗi chuỗi là một đường đi **xuống** từ head của nó.

Chứng minh: mỗi đỉnh v ≠ gốc có đúng một cạnh lên cha. Nếu cạnh đó nặng, v nối dài chuỗi của cha; nếu nhẹ (hoặc v là gốc), v là head của một chuỗi mới. Từ mỗi đỉnh có **nhiều nhất một** cạnh nặng đi xuống, nên chuỗi không rẽ nhánh: nó là một đường đi xuống.

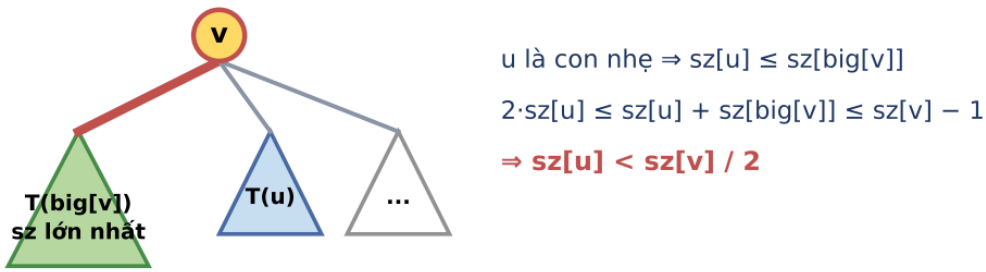
Lê Thành Nam

Định lí cạnh nhẹ

ĐỊNH LÍ 1

Nếu (v, u) là cạnh nhẹ (u là con nhẹ của v) thì **sz[u] < sz[v] / 2**.

Ý 1 So sánh với con nặng



Hình 1.2. *So sánh với con nặng*

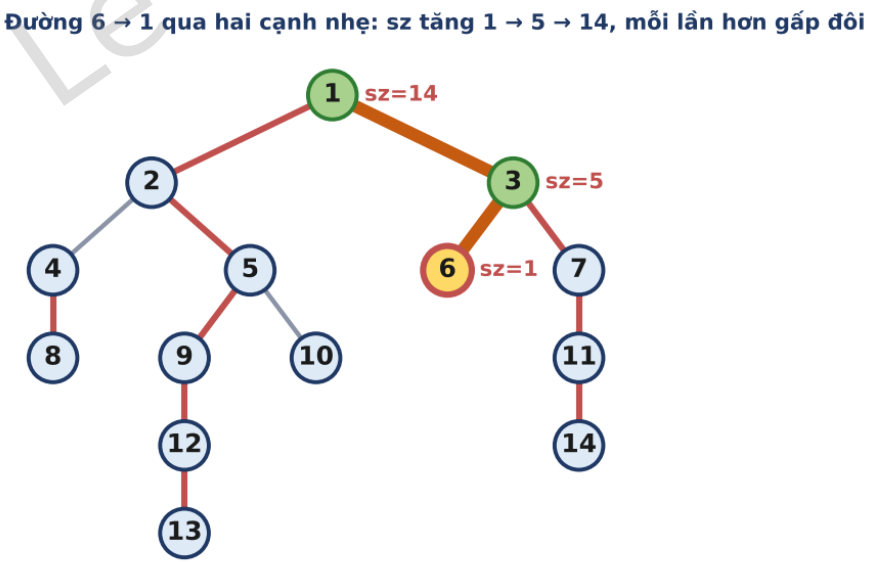
u là con nhẹ nên sz[u] ≤ sz[big[v]] (con nặng có sz lớn nhất). T(u) và T(big[v]) rời nhau, cùng nằm trong T(v) nhưng không chứa v, nên sz[u] + sz[big[v]] ≤ sz[v] − 1.

Do đó 2·sz[u] ≤ sz[u] + sz[big[v]] ≤ sz[v] − 1 < sz[v], tức sz[u] < sz[v] / 2.

ĐỊNH LÍ 2

Trên đường đi từ một đỉnh bất kì lên gốc có **không quá $\log_2 n$ cạnh nhẹ**. Do đó đường đi ấy cắt **không quá $\log_2 n + 1$ chuỗi**.

Ý 1 Mỗi cạnh nhẹ làm sz hơn gấp đôi



Hình 1.3. *Mỗi cạnh nhẹ làm sz hơn gấp đôi*

Lê Thanh Nam

Đi từ x lên gốc, sz của đỉnh đang đứng không giảm. Mỗi lần đi qua một cạnh nhẹ từ u lên v, theo Định lí 1, sz[v] > 2·sz[u]. Bắt đầu với sz[x] ≥ 1, sau k cạnh nhẹ sz > 2^k. Nhưng sz không vượt quá n, nên 2^k < n, tức k < $\log_2 n$.

Đường đi đổi chuỗi đúng tại các cạnh nhẹ (trong một chuỗi chỉ có cạnh nặng), nên số chuỗi bị cắt bằng số cạnh nhẹ cộng 1.

Ví dụ trên Hình 1.3: đường 6 → 3 → 1 qua hai cạnh nhẹ (3 – 6 và 1 – 3), sz tăng 1 → 5 → 14. Với n = $2 \cdot 10^5$, mọi đường đi lên gốc cắt không quá 18 chuỗi.

VÌ SAO PHẢI LÀ CON NẶNG?

Nếu chọn "con nối dài chuỗi" tùy ý, định lí sụp đổ. Trên Hình 1.4, cây hình con sâu bướm: chọn đúng con nặng thì cả xương sống là một chuỗi; chọn nhầm lá thì đi từ cuối xương sống lên gốc phải qua n/2 chuỗi. Chọn theo **chiều sâu** (con có cây con cao nhất) cũng không đảm bảo log n: đó là chia theo đường dài ở Chương 3, đường đi lên gốc có thể cắt tới √(2n) chuỗi.



Hình 1.4. *Chọn con nặng và chọn sai*

A3. Đánh số: chuỗi và cây con đều là đoạn liên tiếp

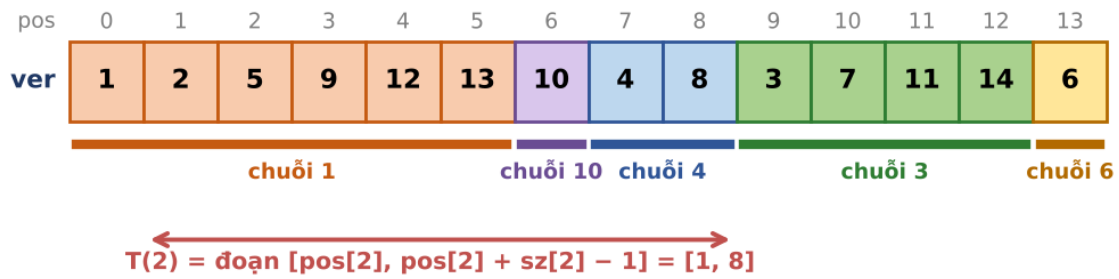
Ta cần đặt các đỉnh vào một mảng sao cho **mỗi chuỗi chiếm một đoạn liên tiếp**, đỉnh nông đứng trước. Cách làm: duyệt **thứ tự trước** (Tập 1) nhưng luôn đi vào **con nặng trước tiên**. Kí hiệu pos[v] là vị trí của v, ver[i] là đỉnh ở vị trí i.

TÍNH CHẤT 2

1. Với mọi v có con nặng: pos[big[v]] = pos[v] + 1. Do đó chuỗi có head h và k đỉnh chiếm đúng đoạn [pos[h], pos[h] + k − 1].
2. Như mọi thứ tự trước: T(v) chiếm đúng đoạn [pos[v], pos[v] + sz[v] − 1].

Chứng minh: (1) ngay sau v, thứ tự trước đi vào con đầu tiên, mà con đầu tiên là con nặng. Lặp lại dọc chuỗi, các đỉnh của chuỗi nhận các số liên tiếp. (2) Thứ tự trước liệt kê xong toàn bộ T(v) rồi mới rời khỏi v, nên T(v) là một khối liên tiếp gồm sz[v] vị trí bắt đầu từ pos[v].

Lê Thành Nam



Hình 1.5. *Mảng ver theo thứ tự trước ưu tiên con nặng*

Trên Hình 1.5: chuỗi cam ở vị trí 0..5, chuỗi xanh lá 3 – 7 – 11 – 14 ở vị trí 9..12. Tính chất (2) sẽ cần ở Bài 3; ở bài này ta chỉ dùng tính chất (1).

Cài đặt không đệ quy

Ba bước, không bước nào đệ quy: (a) BFS từ gốc lấy par, dep và thứ tự BFS; (b) duyệt ngược thứ tự BFS để cộng sz lên cha và chọn big; (c) duyệt thứ tự trước bằng ngăn xếp: lấy v ra, gán pos[v], đẩy các **con nhẹ trước**, đẩy **con nặng sau cùng**. Ngăn xếp lấy ra phần tử đẩy sau cùng trước, nên con nặng được thăm ngay sau v. Đồng thời gán head: con nặng kế thừa head của cha, con nhẹ tự làm head.

Code mẫu: Đánh số pos và head

```
vector<int> head(n + 1), pos(n + 1), st = {1};
head[1] = 1;
int timer = 0;
while (!st.empty()) {
    int v = st.back();
    st.pop_back();
    pos[v] = timer++;
    for (int u : adj[v])
        if (u != par[v] && u != big[v]) {
            head[u] = u;
            st.push_back(u);
        }
    if (big[v]) {
        head[big[v]] = head[v];
        st.push_back(big[v]);
    }
}
```

Lê Thành Nam

A4. Nhảy chuỗi

Có pos và head, đường đi x – y được cắt thành các đoạn như sau.

NHẢY CHUỖI (x, y)

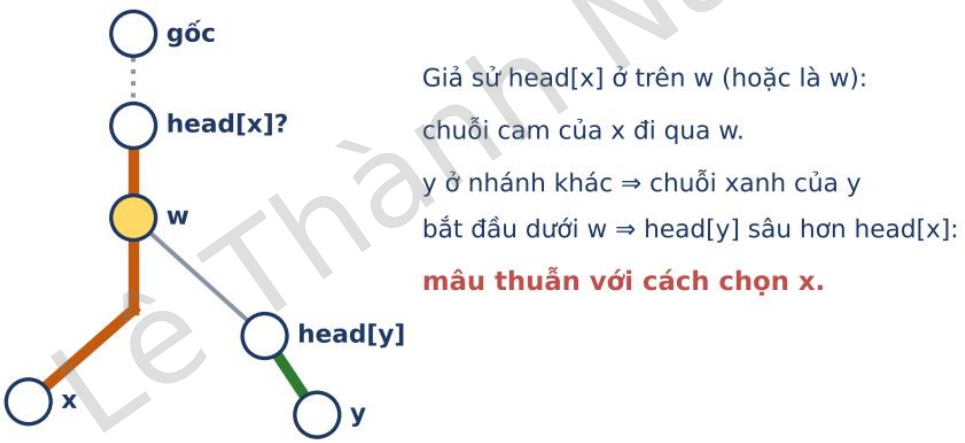
1. Trong khi head[x] ≠ head[y]: nếu dep[head[x]] < dep[head[y]] thì đổi chỗ x và y. Xử lí đoạn **[pos[head[x]], pos[x]]**, rồi gán x = par[head[x]].
2. Lúc này x và y cùng chuỗi. Nếu dep[x] > dep[y] thì đổi chỗ. Xử lí đoạn **[pos[x], pos[y]]**. Đỉnh x lúc này chính là LCA(x, y) ban đầu.

Bước 1 luôn "nhảy" đầu có **head sâu hơn**. Đây là điểm mấu chốt: chọn sai đầu có thể nhảy vượt qua LCA.

ĐỊNH LÍ 3 (ĐÚNG ĐẮN)

Gọi w = LCA(x, y). Ở mỗi lần lặp của bước 1, head[x] (sau khi đổi chỗ) là **hậu duệ thực sự** của w. Do đó đoạn [pos[head[x]], pos[x]] là một mẩu của đường x – w không chứa w, và par[head[x]] vẫn nằm trên đường x – w.

Ý 1 Giả sử ngược lại



Hình 1.6. *Giả sử ngược lại*

Giả sử head[x] là tổ tiên của w hoặc chính là w. Chuỗi của x đi xuống từ head[x] tới x, mà w nằm giữa (w là tổ tiên của x), nên **w thuộc chuỗi của x**.

Xét y. Nếu y thuộc chuỗi của x thì head[y] = head[x], trái với điều kiện vòng lặp. Vậy y nằm trong T(w) nhưng không trên chuỗi của x. Chuỗi của y không chứa w (mỗi đỉnh chỉ thuộc một chuỗi, w đã thuộc chuỗi của x), nên head[y] là hậu duệ thực sự của w: dep[head[y]] > dep[w] ≥ dep[head[x]].

Điều này mâu thuẫn với việc ta đã đổi chỗ để có dep[head[x]] ≥ dep[head[y]].

Ý 2 Kết luận

Mỗi lần lặp lấy đi một mẩu x → head[x] nằm hẳn dưới w, rồi x đi lên par[head[x]], vẫn là hậu duệ (hoặc chính là) w. Các mẩu không giao nhau. Khi hai đầu cùng chuỗi, chuỗi ấy chứa w (cả

Lê Thành Nam

hai đầu đều là hậu duệ của w và cùng một đường đi xuống), đỉnh nông hơn là w, đoạn cuối nối hai đầu qua w. Hợp các mẩu đúng bằng đường x – y.

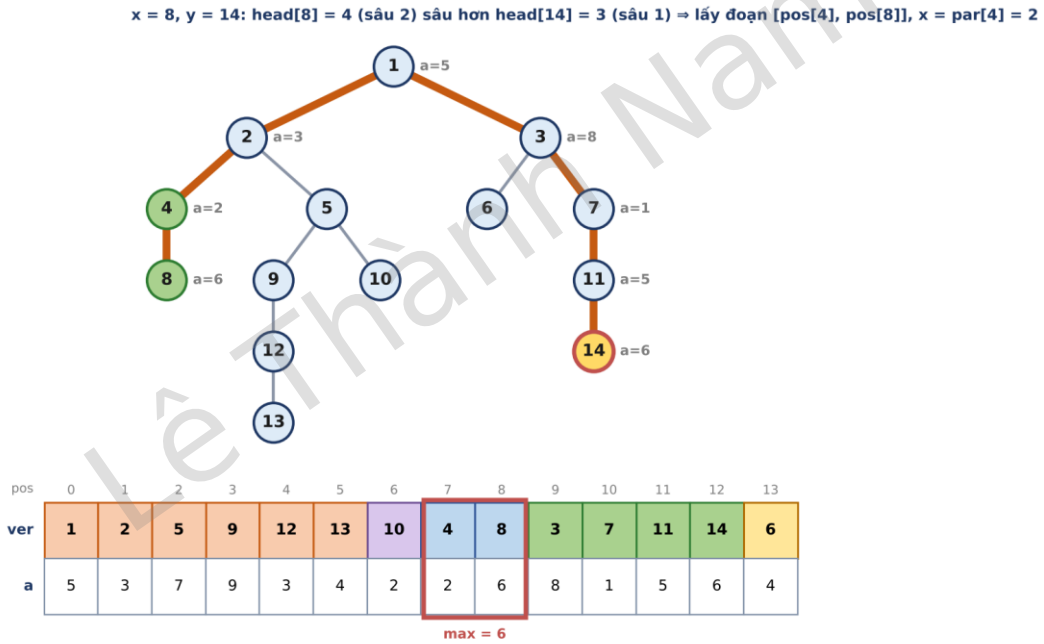
ĐẾM SỐ PHÉP TÍNH

- Mỗi lần lặp ở bước 1, đầu bị nhảy đi qua một cạnh nhẹ (par[head[x]] – head[x]). Theo Định lí 2, mỗi đầu đi qua không quá $\log_2 n$ cạnh nhẹ: tổng không quá **$2 \cdot \log_2 n + 1$ đoạn**.
- Mỗi đoạn hỏi cây phân đoạn O(log n): một câu hỏi đường đi tốn **$O(\log^2 n)$**; đổi giá trị một đỉnh O(log n).

Chạy tay: max trên đường 8 – 14

Giá trị a[v] ghi cạnh mỗi đỉnh (dùng lại ở ví dụ của Phần B). Đường 8 – 4 – 2 – 1 – 3 – 7 – 11 – 14 tô cam.

Bước nhỏ 1 **x = 8, y = 14**

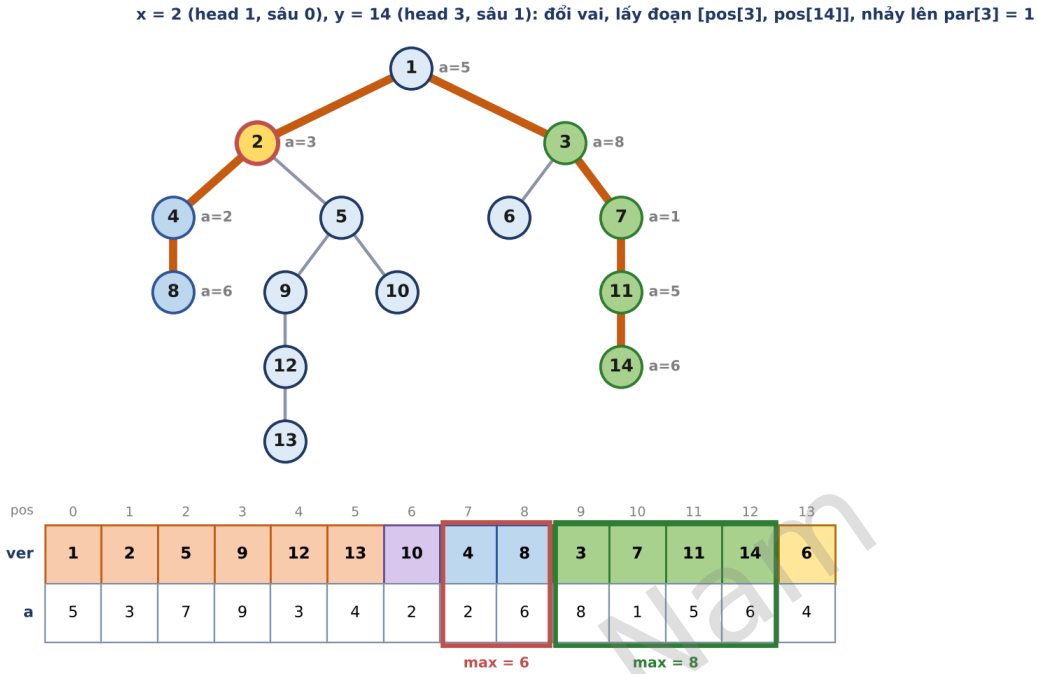


Hình 1.7. *x = 8, y = 14*

head[8] = 4 có độ sâu 2, head[14] = 3 có độ sâu 1: nhảy đầu x. Đoạn [pos[4], pos[8]] = [7, 8] gồm đỉnh 4, 8, max = 6. x = par[4] = 2.

Lê Thành Nam

Bước nhỏ 2 **x = 2, y = 14**

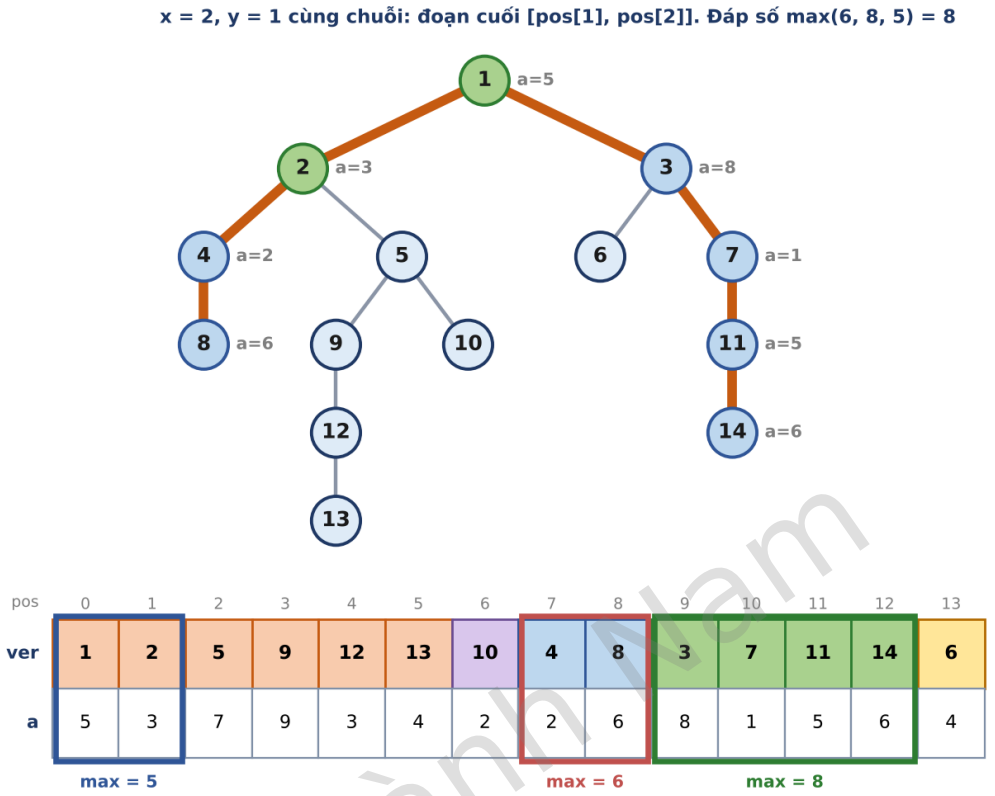


Hình 1.8. *x = 2, y = 14*

head[2] = 1 (độ sâu 0) nông hơn head[14] = 3: đổi chỗ, nhảy đầu 14. Đoạn [9, 12] gồm 3, 7, 11, 14, max = 8. Đầu ấy lên par[3] = 1.

Lê Thanh Nam

Bước nhỏ 3 **Hai đầu 2 và 1 cùng chuỗi**



Hình 1.9. Hai đầu 2 và 1 cùng chuỗi

Đỉnh nông hơn là 1 = LCA(8, 14). Đoạn cuối [pos[1], pos[2]] = [0, 1], max = 5. Đáp số max(6, 8, 5) = **8**.

Thêm một ví dụ, đường 13 – 6: head[6] = 6 (sâu 2) sâu hơn head[13] = 1, lấy đoạn [13, 13] rồi lên 3; head[3] = 3 (sâu 1) sâu hơn head[13] = 1, lấy [9, 9] rồi lên 1; 1 và 13 cùng chuỗi, đoạn cuối [0, 5]. Ba đoạn.

LCA bằng chia nặng nhẹ

Bỏ phần xử lí đoạn, vòng nhảy chuỗi trả lời LCA trong O(log n), bộ nhớ O(n), hằng số rất nhỏ. Đây là cách thứ tư tìm LCA, bên cạnh bảng nhảy, Euler tour + RMQ (Tập 2):

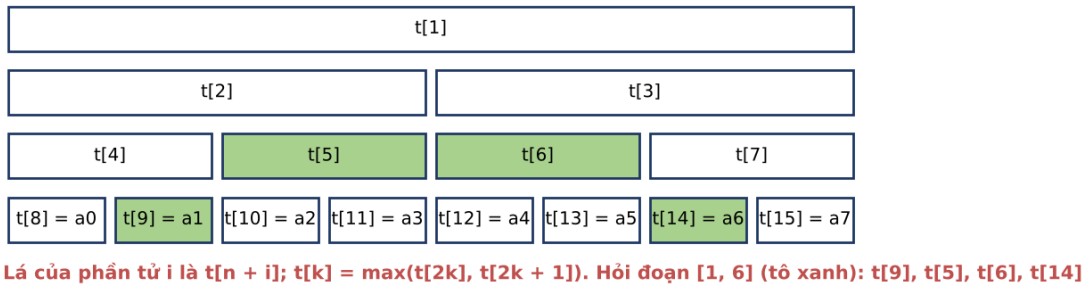
Code mẫu: LCA bằng HLD

```
auto lca = [&](int x, int y) {
    while (head[x] != head[y]) {
        if (dep[head[x]] < dep[head[y]]) swap(x, y);
        x = par[head[x]];
    }
    return dep[x] < dep[y] ? x : y;
};
```

Lê Thành Nam

A5. Cây phân đoạn từ dưới lên

Mảng t có 2n phần tử: lá của vị trí i là t[n + i], nút trong k giữ max(t[2k], t[2k + 1]). Gán một vị trí: sửa lá rồi đi lên bằng phép chia 2. Hỏi đoạn [l, r]: hai con trỏ l, r + 1 cùng đi lên; mỗi khi l lẻ thì t[l] là một mảnh nằm trọn trong đoạn, lấy rồi tăng l; mỗi khi r lẻ thì lấy t[r − 1].



Hình 1.10. *Cây phân đoạn từ dưới lên với n = 8*

Code mẫu: Gán và hỏi max

```
auto gan = [&](int i, int x) {
    for (t[i += n] = x, i /= 2; i >= 1; i /= 2) t[i] = max(t[2 * i], t[2 * i +
1]);
};
auto hoi = [&](int l, int r) {
    int res = 0;
    for (l += n, r += n + 1; l < r; l /= 2, r /= 2) {
        if (l & 1) res = max(res, t[l++]);
        if (r & 1) res = max(res, t[--r]);
    }
    return res;
};
```

Cách viết này không đệ quy, ngắn, và đúng với mọi n (không cần n là lũy thừa của 2) khi phép gộp giao hoán như max, cộng.

BẪY THƯỜNG GẶP

- Nhảy đầu có **x sâu hơn** thay vì **head[x] sâu hơn**: có thể nhảy vượt LCA, kết quả sai.
- Đánh số pos bằng BFS hoặc thứ tự trước bình thường: chuỗi không liên tiếp.
- Đẩy con nặng vào ngăn xếp **trước** con nhẹ: con nặng bị thăm sau cùng, chuỗi đứt.
- Quên đoạn cuối ở bước 2, hoặc lấy đoạn [pos[y], pos[x]] khi pos[x] < pos[y].
- Cây phân đoạn chỉ số theo đỉnh thay vì theo pos.

Lê Thanh Nam

GHI NHỚ

- Con nặng = con có sz lớn nhất; cạnh nhẹ làm sz hơn gấp đôi ⇒ ≤ $\log_2 n$ cạnh nhẹ trên đường lên gốc.
- Thứ tự trước, con nặng trước: chuỗi liên tiếp, cây con liên tiếp.
- Nhảy đầu có head sâu hơn; đoạn [pos[head[x]], pos[x]]; cuối cùng đoạn giữa hai đầu cùng chuỗi.
- Câu hỏi đường đi $O(\log^2 n)$, cập nhật đỉnh $O(\log n)$.

Lê Thành Nam